

Technical report — NLI over Swiss federal voting booklets with Apertus

What we built, how it works, and what the numbers say. Every number comes from a recorded run in docs/runs/ (short names in brackets, listed under References) on the dev split, the validation set (val) or both; **the test split has not been run**.

- **Track:** Track 2A – OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Online
- **Team:** TODO team name — Rashad, Kaan (TODO full names)
- **Demo:** none

1. Summary

Given a claim in German, French or Italian and a source from an official Swiss federal voting booklet, the system decides whether the source supports (entailment, 0), does not settle (neutral, 1) or refutes (contradiction, 2) the claim. The source is a passage (task B) or a whole booklet PDF (task A). **Apertus v1.5 makes every decision**. Everything else is local, deterministic and runs on CPU inside the Docker image: PDF parsing, choosing what part of the booklet Apertus reads, and turning the paragraphs Apertus cites into evidence quoted verbatim with their 1-based PDF page.

The central experiment, full booklet against selected context, decided the design: with the same prompt, sending only the vote's pages instead of the whole booklet raised task A's Macro-F1 from 0.669 to 0.732 at 40 % of the tokens [E2]; sending only the part of the vote that the claim names (section-route) reached 0.953 at 3 % of the tokens [E5]. In one interleaved run on 100 dev cases [E1-curve], the whole booklet scored 0.858 with 37,586 input tokens per case, the routed part 0.989 with 1,248; without any booklet text (claim and vote name only) Apertus scores 0.435 [E2-cb]: the context, chosen well, carries the result.

Final defaults	Task A Macro-F1	Task A evidence	Task B Macro-F1	Input tokens / case (A, B)	Mean / p95 time (A; B)
dev, 600 cases, Docker image [FIN]	0.980	0.980	0.967	1,238, 1,231	2.2 / 4.5 s; 2.0 / 3.8 s
val, 580 rows [D-val, C-val]	0.961	0.958	0.961	1,206, 1,230	1.9 / 3.0 s; 1.8 / 3.8 s

Session 9 changed three defaults, each by a rule fixed before its interleaved runs and confirmed on val: evidence adds halves of the cited paragraphs (A2), task B cuts long references (B-cut, -37 % tokens at equal Macro-F1) and task A's prompt names what makes a contradiction (L1, +0.010 to +0.014). Both tasks are far above the starter's minimum for a valid submission (task A 0.60, task B 0.75). The private test set may differ (section 6).

2. Architecture

cases.jsonl → src/cli.py

task B: reference text → B-cut if over 8,000 characters → prompt

task A: booklet PDF → src/parse.py (pypdf, text per 1-based page)

└ section-route: src/claim_router.py (part named by the opening)

└ + src/booklet.py (votes, parts) → numbered paragraphs → prompt

└ fallback: embed-e5-small (8 most similar chunks) → prompt

src/llm.py → BASE_URL (Apertus v1.5): label + cited paragraphs

predictions.jsonl ← evidence: cited paragraphs, verbatim, with page

- src/cli.py: the official entrypoint (--input, --output). Answers every request that has an id, never stops on one bad case (fallback: neutral, no evidence), writes each answer at once, exit code 0. Reads the input as bytes line by line (bad bytes, byte order mark, all line endings), answers a repeated id once.

- `src/llm.py`: the only module that calls a model: one OpenAI-style chat/completions request per case to `BASE_URL` with `API_KEY`, model from `MODEL`, temperature 0. One retry for HTTP 5xx and timeouts, two for HTTP 429. If the endpoint refuses `response_format` (HTTP 400/422) it resends without it; if it refuses the model name it reads `BASE_URL/models` once and takes the Apertus v1.5 8B id (at most three extra requests per run). Tokens of every request are summed into the case's metrics, and every wait counts in its time.
- `src/nli.py` (prompts, versioned by name; answer parsing); `src/context.py` and `src/contexts/` (task A context variants, one file each); `src/booklet.py`, `src/claim_router.py` (parser and router); `src/taskb_context.py` (task B passages).

Everything runs in one container (linux/amd64, CPU only); the only network traffic is the call to `BASE_URL`.

2.1 How document context is prepared and supplied to the model

Parsing. `pypdf` reads the PDF locally and keeps each page's text with its 1-based PDF page number (the n -th page of the file, not the printed number). Results are cached in `/tmp` under the file's SHA-256.

Selection (section-route, the task A default).

1. *Router*: the dataset's claims open with a formula naming their source ("Laut der Zusammenfassung ...", "Le Conseil fédéral estime ...", "Il comitato afferma ...", "Laut dem Abstimmungstext ...", "Si le vote est accepté ..."). Regular expressions on the opening map it to one of five parts: summary, Federal Council, committee, text put to the vote, detailed explanation; or to nothing.
2. *Parser*: the table of contents gives each vote's sections; every start page is checked against the heading printed on it; the arguments are split into the committee's and the Federal Council's, the summary's recommendation boxes per voice. A vote that fails a check gets no parts (the case falls back).
3. *Vote match*: the case's vote against the contents titles (fuzzy partial ratio ≥ 80 , 5 points clear).
4. *Paragraphs*: the part's lines grouped into paragraphs (running headers, page numbers and fragments under 40 characters dropped), each with its page. A part over 8,000 characters keeps the 8 paragraphs most similar to the claim (multilingual-e5-small), in page order.
5. *Fallback*: no route, no vote, or any error $\rightarrow$ the case runs as `embed-e5-small` (the 8 chunks of at most 1,000 characters most similar to the claim, each after `=== PAGE n ===`). 1 of 300 dev cases falls back.

What the prompt contains. System: the instruction and the decision rule (word for word in `src/nli.py`, A-v4-section-route):

Decide in this order: 1. Does the reference text deal with the subject of the claim at all? If not, the label is 1 (neutral). 2. If it does: 0 (entailment) if the reference text supports the claim; 2 (contradiction) only if the reference text states something that cannot be true together with the claim; otherwise 1 (neutral: insufficient information). Missing information is never a contradiction. [...] A claim that gives a different number, share, date, actor or direction than the reference text gives for the same thing is a contradiction.

(The last sentence since session 9, phase D: A-v4-section-route-L1.)

User: PART: (which part, whose voice), REFERENCE TEXT: with paragraphs [1] ... [n], VOTE:, CLAIM:. Nothing is translated. The answer is forced by `response_format: json_schema` to `{"paragraphs": [ $\leq 3$  numbers], "label": 0|1|2}`. Task B sends the same rule (v3-topic-first) with REFERENCE TEXT: and CLAIM: and asks for `{"label": n}` in the prompt only.

Evidence. Only what Apertus cited (team decision): the cited paragraphs verbatim with their page, then the two halves of each cited paragraph (cut at the sentence end nearest the middle) until there are five items; a repeated text is left out. Neutral answers and task B carry no evidence.

3. Use of Apertus

- **Model:** Apertus v1.5, named by `MODEL` (default `swiss-ai/Apertus-v1.5-8B`). Our runs used `swiss-ai/apertus-v1.5-8b` on Public AI (a few early runs ...-8b-thinking). Which Apertus model and server the evaluation uses is not known to us.
- **How it is used:** inference only, zero-shot, one call per case, temperature 0; answer budget 128 tokens (task A) and 32 (task B). No fine-tuning.

- **Where it runs:** a hosted OpenAI-compatible endpoint reached only through `BASE_URL`. No key in the image.
- **Apertus makes every entailment decision.** The local embedding model (multilingual-e5-small, MIT, open weights, ONNX, baked into the image, CPU via onnxruntime) only chooses which paragraphs Apertus reads.
- **Labels:** the dataset README (checked 2026-10-09) names the classes only: “May be Entailment (0), Unrelated / Neutral (1) or Contradiction (2)”. The prompts use the official guide’s wording (supports, insufficient information, refutes), which agrees with these names.
- **Development tools:** Claude Code wrote code and ran experiments. It is not part of the solution and is never called by the pipeline.

4. Data

- **Dataset:** OSTswiss/MNLIOverSwissVotingBooklets v1.1 (commit 9ff08597, MIT): 1,488 rows, 335 exact duplicates; each row gives one task A and one task B case.
- **Splits** (by voting date, seed 42, duplicates removed): test 5 dates, 267 rows, **never run**; dev 300 rows from the other 15 dates, balanced over labels and the nine language pairs; val: the 580 remaining rows of the dev dates (minus six used to write router patterns), new claims, never used to change a rule.
- **Booklets:** the Federal Chancellery’s PDFs, downloaded by script and not committed (one example booklet for make run). admin.ch’s terms say any reproduction requires prior written consent; Swiss copyright law exempts official reports of authorities (Art. 5 URG). Whether the example booklet may stay is for the team to settle. No personal data.

5. Evaluation

Method. The starter’s `evaluate.py` (commit 559b598), unchanged: Macro-F1 per task from labels; for task A the evidence score (share of gold entailment/contradiction cases where one of the first five items fuzzy-matches the gold passage). Tokens and time are the pipeline’s per-case metrics (the endpoint’s usage; wall-clock per case). The endpoint drifts (same prompt, temperature 0, different answers on different days and backends, section 6), so **only arms of one interleaved run compare**: all versions answer each case back to back, in rotating order, and every answer records which backend wrote it. Session 9’s adoption rules were fixed before the runs; a version that failed stays in the code, switched off.

Task A, from the baseline to the default (dev, 300 cases):

Setup [run]	Macro-F1	Evidence	Input tokens	Mean / p95 time
Whole booklet, JSON asked in the prompt (110 answers unreadable) [S2]	0.589	0.209	39,706	11.5 / 30.3 s
Whole booklet, json_schema [E2]	0.669	0.284	39,206	12.7 / 37.1 s
Vote’s pages, json_schema [E2]	0.732	0.224	15,868	8.2 / 13.5 s
embed-e5-small [E5]	0.834	0.662	1,868	3.2 / 11.7 s
section-route [E5]	0.953	0.905	1,210	2.0 / 3.9 s
val sample: embed-e5-small / section-route [E6]	0.865 / 0.956	0.588 / 0.946	1,827 / 1,222	3.2 / 1.8 s

Context curve (E1, one interleaved run, 100 balanced dev cases, defaults of session 9) [E1-curve]:

What Apertus reads	Macro-F1	Evidence	Input tokens	Mean / p95 time
Whole booklet	0.858	0.422	37,586	7.3 / 17.6 s
8 most similar chunks of the booklet (embed-e5-small)	0.898	0.656	1,885	4.9 / 15.3 s
The routed part (section-route)	0.989	1.000	1,248	1.9 / 3.0 s
Its 4 / 2 / 1 most similar paragraphs	0.979 / 0.940 / 0.769	0.984 / 0.922 / 0.641	804 / 614 / 510	1.8-2.0 s
Nothing: claim and vote name only [E2-cb, 300 cases]	0.435 (section-route 0.980)	-	223	1.5 s

Session 9 experiments (interleaved runs; “same backend”: only cases where all arms met one backend):

Experiment [run]	Versions	Macro-F1 (all / same backend)	Input tokens	Rule, outcome
Task B long passages, dev [C-dev]	current / B-cut / B-para	0.967 / 0.967 / 0.963 (all one backend)	1,994 / 1,231 / 1,375	B-cut $\leq$ 0.01 below, $\geq$ 25 % fewer tokens: passes
B-cut confirmed on val, 580 cases [C-val]	current / B-cut	0.957 / 0.961 (all one backend)	1,957 / 1,230	same rule holds: default since session 9
Task A label errors, dev [D-dev]	current / L1 / L2	0.966 / 0.980 / 0.976 (all one backend)	1,210 / 1,238 / 1,244	L1 +0.01 and neutral recall $\geq$ 0.98: passes; L2 +0.015: fails
L1 confirmed on val, 580 cases [D-val]	current / L1 / (L2)	0.950 / 0.961 / 0.954 (all one backend)	1,178 / 1,206 / 1,217	same rule holds for L1: default since session 9

Error margin and languages. A 95 % bootstrap interval of task A’s Macro-F1 on dev and val pooled (600 cases) is 0.938–0.970; with 300 cases, differences of 0.02–0.03 between separate runs are within sampling noise alone. No language pair stands out (task A accuracy per claim/booklet pair 0.90–0.99, about 67 cases each; task B errors are all cross-language, 10 of 200).

Where the errors are. All 12 routed errors of E6 (val) had the gold passage among the paragraphs sent, and 10 of them cited a paragraph inside it: reading errors, not search errors. 9 of the 13 call a supported or refuted claim neutral. Phase D targeted them: one more sentence in the prompt (L1) cut the contradictions answered neutral from 7 to 2 on dev and 21 to 14 on val, without a single neutral case lost; a second look at neutral answers similar to the text (L2) fixed fewer (3 on dev, 2 on val) for 3 % more tokens.

Evidence pages. Every evidence item’s page is the page that holds its quote; 179 of 182 dev items that match the gold passage lie on one of its pages, 107 on its first page (a passage often spans pages).

Token usage and inference time (final defaults, all 600 dev cases through the image [FIN], per case; time is wall-clock per case including local work; the model’s own share is not separable):

Task	Input tokens (mean / p95)	Output tokens (mean)	Model calls	Mean / p95 time
A	1,238 / 2,132	14.5	1	2.2 / 4.5 s
B	1,231 / 2,182	8.4	1	2.0 / 3.8 s

Since session 9, task B sends a reference over 8,000 characters as its first line plus the 8 paragraphs most similar to the claim (B-cut): 37–38 % fewer input tokens at the same Macro-F1, for about 0.3 s more per case (the e5 embedding of the long reference on the CPU). Time includes local work: without the model, a task A case takes milliseconds once its booklet is parsed; the first case on a booklet pays up to 3.5 s for parsing, and long parts are embedded (slowest case 17.6 s on 2 CPUs, peak memory 2.2 GiB).

Stability. The same image (built before session 9’s default changes) and settings on all 600 dev cases at three times [S1, S2, S3]:

Point (UTC)	Task A Macro-F1	Task B Macro-F1	Labels equal to point 1 (A; B)
1, 10-09 17:28	0.966	0.967	–
2, 10-10 01:56	0.966	0.967	299/300; 300/300
3, 10-10 {}	{}	{}	{}

6. Limitations

- **Tuned on dev.** Prompts, router patterns and parser checks were written with the dev data in view; val (new claims, same booklets) confirms the results; unseen booklets were checked offline only (14 of 15 booklets from 2018–2026 parse completely); the test split and the private set have not been run.
- **section-route depends on the claims’ openings.** A claim that names its source differently falls back to embed-e5-small (still answered, less accurate). On 300 reworded openings written for a stress test, 273 are routed right and none to a wrong part; the patterns were written after seeing them.
- **The parser depends on the booklet layout** (2018–2026 checked). A vote that fails a check falls back.
- **Endpoint drift and backends.** Public AI served swiss-ai/apertus-v1.5-8b from two backends on 2026-10-09 (one named Apertus-8B-Instruct-2509, the v1 release), and the backend changes the answers: the same task B prompt scored 0.966 on one and 0.566 on the other within one run [B4arm]. Our scores hold for the backend that answered; the evaluation’s model and server are unknown.
- **A dataset artefact we do not use.** Task B references over 8,000 characters are 99 of 105 times neutral on dev, shorter ones never: neutral rows pair the claim with a whole unrelated section. A length rule would score 0.973 on dev task B but says nothing about the claim; the pipeline never uses it.
- **Evidence pages:** the starter does not check pages; the official scorer might, with an unknown rule.
- **Not political advice.** The system checks a claim against the booklet’s text only. Its answers are not voting recommendations. Every task A label 0 or 2 comes with the booklet page and the verbatim text it rests on, so it can be traced to the source booklet.

7. Reproducibility

- **Run:** make run builds the image and answers examples/cases.jsonl; BASE_URL, API_KEY and optionally MODEL from the environment. Own cases: make run CASES=... BOOKLETS=... OUTPUT_DIR=... (README). Dev: python3 scripts/fetch_dev_booklets.py, then make run CASES=data/dev/cases.jsonl BOOKLETS=output/booklets_dev OUTPUT_DIR=output/dev; score with the starter’s evaluate.py.
- **Hardware:** CPU only; image 1.29 GB; peak memory 2.2 GiB with 2 CPUs and 4 GB; the evaluation’s hardware is unknown to us.
- **Determinism:** temperature 0, local steps deterministic, caches keyed by content, split seed 42. Every run folder names its commit. The endpoint’s drift is outside our control.
- **Checks without a model:** a fake model (scripts/stub_llm.py) replays saved answers by request hash; for every code change, the requests of all 600 dev and 580 val cases and the replayed labels are compared with a reference (gates G1, G2); a clean-machine workflow builds the image on a fresh runner and runs it offline with a read-only /data.

8. Next steps

- Ask the organisers which model, server and page rule the evaluation uses, and whether Public AI's two backends (one serving the v1 release under the v1.5 name) will be in play.
- Cut routed parts to their 4 most similar paragraphs: −36 % tokens for −0.01 Macro-F1 on 100 dev cases (E1); needs its own interleaved run on dev and val.
- Ask Apertus to route only when the rules find no part (E3: it routes all 15 openings the rules miss, but also picks parts for openings that name none).
- L2 as an option if tokens matter less than the last errors; a smaller image (P11, P12).

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. The repository's LICENSE file is Apache-2.0; the team has yet to make the two agree.

References

- Dataset: OSTswiss/MNLloverSwissVotingBooklets v1.1, Hugging Face (MIT). Starter: gitlab.com/ifsoftware/hackapertus-starter, commit 559b598. Apertus v1.5, Swiss AI Initiative.
- Wang, L. et al. (2024). *Multilingual E5 Text Embeddings: A Technical Report*. arXiv:2402.05672 (intfloat/multilingual-e5-small, MIT). pypdf (BSD-3), rapidfuzz (MIT), onnxruntime (MIT).
- Runs (docs/runs/): S2 s2-A300-A-v3-fulldoc; E2 s3-E2-A300; E5 2026-10-08_rashad_section-route-vs-embed_devA300; E6 2026-10-09_rashad_section-route-vs-embed_valA300; B4arm 2026-10-09_rashad_taskb-4arm_devB300; S1 2026-10-09_rashad_stability-1_dev600; C-dev 2026-10-09_rashad_taskb-context_devB300; C-val 2026-10-09_rashad_taskb-cut-confirm_valB580; D-dev 2026-10-09_rashad_label-errors_devA300; D-val 2026-10-09_rashad_label-errors-confirm_valA580; E1-curve 2026-10-09_rashad_context-curve_devA100; E2-cb 2026-10-09_rashad_closed-book_devA300; E3-router 2026-10-09_rashad_llm-router_dev300-stress300; FIN 2026-10-09_rashad_final-defaults_dev600; S2, S3 2026-10-10_rashad_stability-{2,3}_dev600.